

Top 25 Low-Level Design (LLD) Interview Questions for Entry-Level SDEs (Amazon/Meta/Google)

Low-Level Design (LLD) interview questions focus on object-oriented design of systems, evaluating how well you can define classes, relationships, and interactions to solve a problem. These questions are common in entry-level SDE interviews at companies like Amazon (which often includes an OOP design round for new grads), and sometimes at Meta or Google. Below is a ranked list of the 25 most frequently asked LLD questions, with the most common appearing first. For each design problem, we outline what the question entails and what interviewers expect in an answer, including key features to discuss and depth of detail.

1. Design a Parking Lot System

Why it's asked: This is widely considered *the* most common LLD interview question, especially at Amazon ¹. It tests understanding of classes, relationships (composition/inheritance), and basic design patterns.

What to cover:

- **Core Features:** Multiple levels/floors of parking, ability to **park** and **unpark** vehicles, track available spots, and perhaps search for a vehicle by license number ¹.
- **Entities:** Classes like `ParkingLot`, `ParkingFloor`, `ParkingSpot`, and `Vehicle` (with subclasses for different vehicle types). Each floor contains a collection (e.g. 2D array or list) of parking spots. Spots can be of various sizes/types (compact, large, handicapped, etc.).
- **Behavior:** Discuss how to assign an incoming vehicle to an appropriate spot (e.g. smallest available spot that fits). Mention updating counts of free spots on each floor and overall. The design should prevent parking when at full capacity.
- **Design Patterns:** Because there can be different parking strategies (e.g. nearest spot vs. random), using a Strategy pattern for the parking assignment can make the system extensible. Also consider using Singleton for a central `ParkingLot` instance (if needed) and possibly an Observer for display boards to show free spots (if such features are mentioned).
- **Depth:** Interviewers expect a class diagram-level discussion: describe classes, their attributes (e.g. `ParkingSpot` has a spot ID, type, and occupancy status) and methods (e.g. `parkVehicle()`, `leaveSpot()`). You should handle basic scenarios like parking when full, or vehicles leaving and freeing a spot. In Amazon interviews, this question often comes with follow-up questions or slight twists (like adding electric vehicle charging spots or payment calculation), so be prepared to discuss extensions.

2. Design an LRU Cache

Why it's asked: Amazon frequently asks candidates to design and implement an LRU (Least Recently Used) Cache as a low-level design or "machine coding" problem. It evaluates your ability to design efficient data structures with proper class design and OOP principles (and often to write code for it). Google and Meta also value understanding of caching strategies, though Amazon is particularly known for this.

What to cover:

- **Objective:** Create a cache with a fixed capacity that supports `get(key)` and `put(key, value)` operations, where getting/putting moves items to the “recently used” end and evicts the least recently used item when capacity is exceeded.
- **Data Structures:** Explain that an **O(1)** implementation is expected. Common solution: use a doubly-linked list to track usage order and a hash map for fast key lookups. For instance, design classes or structures for `CacheNode` (with key, value, pointers) and `LRUCache` which contains the map and the head/tail of the list.
- **Behavior:** On `get(key)`, return the value if present and update it to most recent (move node to tail of list). On `put(key, value)`, if key exists update value and move node to most recent; if new key, insert at most recent end and if over capacity, remove the least used node from the list (head) and delete it from the map.
- **Class Design:** You might not need multiple classes beyond the cache and node, but ensure you encapsulate the logic well. In Python, for example, you might talk about using `OrderedDict` from `collections` to simplify, but in an interview you'd likely implement the list manually to demonstrate understanding.
- **Depth:** Interviewers expect you to write or describe actual code for core methods. In a coding round, you would implement `get` and `put`. Emphasize correctness and efficiency. For instance, one candidate described starting with a naive list approach then optimizing to O(1) by introducing a hashmap + linked list. Also mention edge cases: getting a key not present (return -1 or error), or handling capacity 0. This problem tests both design (choosing appropriate structures) and coding, so be prepared for detailed follow-up on your implementation.

3. Design a Movie Ticket Booking System (e.g. BookMyShow)

Why it's asked: This question tests your ability to design a system with multiple interacting entities (movies, theatres, showtimes, bookings) and is commonly asked in tech interviews (it has appeared in companies like Amazon, Microsoft, Flipkart, etc.). It requires handling object relationships and some aspects of concurrency (like seat availability).

What to cover:

- **Core Features:** Allow users to **search** movies and view show times, **book** tickets for a selected show (and choose seats), possibly **cancel** bookings. The system should manage multiple cinemas (theaters) with multiple halls/screens, each playing shows for various movies. Additionally, users might want to see which cinemas in a city are playing a particular movie, or all showtimes for a movie in a given theater ².
- **Entities:** Key classes include `Movie`, `CinemaHall` (or `Theater`), `Show` (showtime), `Seat`, `Booking`, and `User`. A `CinemaHall` contains seats and can schedule multiple `Show` instances (with date/time and the movie being shown). `Booking` would link a user to a specific show and specific seat(s).
- **Relationships:** Discuss how movies are scheduled in halls via shows, and how bookings tie users to seats in a show. You might mention using an aggregation of `Show` objects in `CinemaHall`, and each `Show` having a seating layout or a map of seat availability.
- **Features and Constraints:** Ensure to cover seat selection and locking (to avoid double-booking seats). For instance, once a seat is booked for a show, it should not be available to others. Mention that the system should support listing all cinemas playing a particular movie or all shows of a movie in one cinema ³ – this hints at maintaining some indexes or search capability. - **Design Patterns:** An interesting aspect is updating availability/search results. One approach uses the Observer pattern: for example, when a new show is added to a cinema, notify a “MovieListing” service to update its mapping of movies to cinemas ⁴. This was

suggested as a way to keep the list of shows per movie updated without tight coupling.

- **Depth:** Focus on class design (no need to design the entire database or payment gateway in an entry-level LLD). The interviewer expects discussion of how to model seats (maybe a simple matrix or list in each show), how to prevent race conditions in booking (perhaps mention that concurrency control or transactions would be needed in a real system, though in LLD a simple approach is fine). The complexity can go deeper (like different seat categories, pricing, etc.), but for new grad level, keeping it to basics with a clean OO design is key.

4. Design an Online Shopping System / Shopping Cart

Why it's asked: Designing an e-commerce system (or a part of it like a shopping cart) is a common LLD exercise. Amazon being an e-commerce giant makes this especially relevant. This tests your ability to handle inventory, orders, and user interactions in an object-oriented way ⁵.

What to cover:

- **Scope Clarification:** For an entry-level interview, this design can be scoped to a **Shopping Cart module** or a simplified online bookstore. Clarify whether the focus is on the cart and checkout, or the broader system (product catalog, etc.).

- **Core Features:**

- **Product Catalog & Search:** The system should store products (or books) with details. Users can **search** or browse products by title, category, etc ⁶. Mention classes like `Product` (or `Book`) and possibly a `Catalog` or search service.

- **Shopping Cart:** Users can **add items to cart, remove items, update quantities**. The cart should be associated with a User (or session). Discuss a `ShoppingCart` class that holds a list of `CartItem` (product + quantity).

- **Inventory Management:** Keep track of stock for each product ⁷. For example, an `Inventory` class or simply an attribute in `Product` indicating quantity available. When an order is placed, inventory must decrement; if out of stock, the system should prevent adding to cart or checkout.

- **Order Placement:** Outline the process of **checkout** – creating an `Order` from the cart, processing payment (you can abstract payment as an external service or a simple method call), and confirming the order. An `Order` class would contain order items, shipping info, payment status, etc.

- **Entities and Relationships:** Key classes include `User`, `Product`, `ShoppingCart`, `CartItem`, `Order`, `Payment`, perhaps `Address` for shipping. The `User` has a cart; the cart has items referencing products. An `Order` can be created from the cart.

- **Additional Considerations:** Mention that in an actual system, you'd consider user authentication and maintaining sessions, but in LLD focus on the interactions of objects. Also, note how the design ensures **scalability** – e.g., searching by title might involve efficient data structures or indexes, but at LLD level, you can just mention an in-memory approach or call to a search module.

- **Depth:** Interviewers expect you to handle the basic flow from adding to cart through checkout. Discussing how to prevent inconsistent states (like checking inventory before finalizing an order) is a plus. At entry level, they likely won't require designing a full payment processing system or recommendation system – keep it focused on cart and order classes. Use of design patterns isn't mandatory here, but you could mention using Factory for creating different types of Products or Strategy for different payment methods, for instance, if appropriate.

5. Design a Library Management System

Why it's asked: It's a classic OOD question to test modeling real-world entities and relationships (books, borrowers, etc.) and enforcing constraints like borrow/return limits. This often appears in interviews for its familiarity and depth. Google and Amazon have been known to ask variations of this (sometimes framed as a library or an "online bookstore" system) ⁸.

What to cover:

- **Core Features:** The system manages a collection of books and library members. Key operations include **checking out a book**, **returning a book**, and tracking **due dates** or fines. Additionally, members should be able to search the catalog (by title/author/genre).

- Entities:

- **Book**: with attributes like ISBN, title, author, and whether it's available or on loan.
- **LibraryMember** (or **User**): with details and maybe a limit on how many books can be borrowed at once.
- **Loan** or **BorrowRecord**: representing a book checked out to a member with a borrow date and due date.
- Possibly a **Librarian** or **Admin** class if you consider library staff functions (adding/removing books, etc.).
- **Relationships:** One member can borrow many books (within a limit), and a book can be borrowed by one member at a time. Model this via the **Loan** class or by having a member's account hold a list of borrowed books.

- Key Functionalities:

- **Borrowing:** When a member borrows a book, create a record linking that book to the member and mark the book as not available.
- **Returning:** Free up the book's status and remove or archive the loan record. If implementing due dates, check if the return is late and if so, compute fines.
- **Searching Catalog:** Allow queries like by title or author. A simple approach is to keep lists or use a hashmap keyed by title/author to find books ⁶, but mainly demonstrate that you'd have methods to get books by criteria.
- **Overdue Management:** Optionally mention that if a book is overdue, the system could prevent the member from borrowing more until return or could record a fine ⁹. This shows awareness of real-world constraints.
- **Design Considerations:** Include how to uniquely identify books (ISBN for the title/edition, and maybe a separate ID for each physical copy if multiple copies exist). Ensure your design can handle multiple copies of the same book title.
- **Depth:** Typically, interviewers want to see the class diagram level: classes and main methods (e.g., `checkoutBook(member, book)`, `returnBook(member, book)`). They might ask about edge cases like "what if a book is not available" (you should then refuse the checkout or place a hold request, depending on scope). Solid understanding of basic OOP principles (like not exposing internal lists directly, etc.) will be judged.

6. Design an Elevator (Lift) System

Why it's asked: Elevator systems are a favorite LLD problem to assess handling of real-time state changes and algorithms (for scheduling elevators). Companies like Amazon, Uber, and others have been known to

ask this at entry and mid levels because it combines OOP design with a bit of system behavior complexity

10 .

What to cover:

- **Scenario:** You have a building with multiple elevators. Design the software to move elevators efficiently in response to user requests (calling an elevator to a floor, selecting destination floor inside elevator).

- **Core Features:**

- **State Tracking:** Each elevator has a state: moving **Up**, moving **Down**, or **Idle**. It also has a current floor and a list of pending requests (floors to stop at).

- **Request Handling:** There are two kinds of requests – *external requests* (someone on a floor presses up/down button) and *internal requests* (passenger presses a floor number inside the elevator). The system needs to handle these optimally – e.g., assign the nearest idle elevator or the one already headed that direction (this is the elevator scheduling algorithm) ¹¹ .

- **Scheduling Algorithm:** Discuss a strategy for which elevator answers a call. A simple approach: if an elevator is idle, send it; if not, maybe the one that will pass that floor on its route. Highlight that multiple algorithms exist (look up "SCAN algorithm" or others if familiar, but for entry level, demonstrating logical reasoning is enough).

- **Entities:** Key classes might be `Elevator`, `ElevatorController` (or `System`), and perhaps classes for `Request` or panels. Each `Elevator` object has attributes like `currentFloor`, `direction`, and methods like `moveToFloor(floor)` or `addDestination(floor)`. The controller can manage a list of elevators and dispatch requests to them.

- **Behavior & Design Patterns:** You can use the **State pattern** for the elevator's state machine – e.g., different behavior when Idle vs. Moving (like an Idle elevator can be dispatched immediately, a Moving one might queue requests). Another pattern could be **Observer** – floors (or a central system) notify the elevator of requests, or the elevator notifies the system when it becomes idle. These patterns structure the solution but even without naming them, describe the logic clearly.

- **Depth:** Focus on one elevator first – how it processes a sequence of floor stops (queue of requests), and how you'd ensure it stops at requested floors. Then explain how to scale to multiple elevators with a controller coordinating. Interviewers often ask how to avoid two elevators responding to the same call – mention a centralized dispatcher or locking a request to an elevator. Given the complexity, entry-level interviews expect high-level discussion with maybe some pseudocode for scheduling logic. You likely won't need to write actual code for the whole system, but be ready to draw or describe the class relationships and a flow for handling a new request.

7. Design an Amazon Locker System

Why it's asked: Amazon Locker is a real-world service by Amazon, making it a favorite for Amazon interviews. It tests your ability to design an OOP system that mirrors a physical process (package delivery and pickup) and often comes up in Amazon's new grad or SDE 2 interviews. The question is usually explicitly framed as "*Design Amazon Locker*" and interviewers clarify they expect an object-oriented design (not a high-level distributed system design).

What to cover:

- **System Overview:** Amazon Lockers are secure lockers where couriers can drop off packages for customers to pick up. Focus on designing software for a single locker location (bank of lockers).

- **Core Features:**

- **Locker Allocation:** When a package arrives, the system should assign it to an appropriate locker (that's

free and size-fitting the package).

- **Package Deposit:** The delivery agent can open the assigned locker and place the package. The system should then mark that locker as occupied and generate a pickup code or notification for the customer.

- **Customer Pickup:** A customer arrives, identifies themselves (via code or QR scan), and the system should unlock the correct locker for them to retrieve their package, then mark the locker as free again.

- **Entities:**

- `LockerUnit` : represents a single locker compartment, with an ID, size (e.g. small/medium/large), and status (occupied or free).

- `Package` : with attributes like `packageId`, `size`, `pickupCode`, and maybe an expiration date (how long it can stay).

- `LockerManager` or `LockerSystem` : to manage multiple lockers, find free lockers, and map packages to lockers. It might also handle generating pickup codes and verifying them.

- `User` (or specifically `Customer`) and `DeliveryAgent` classes to represent actors, if needed.

- **Workflow:** Describe how a **package drop-off** works: the delivery agent requests a locker, the system finds an available locker of sufficient size (e.g., using a hash map of size -> list of free lockers), assigns it, and returns the locker info (maybe opens it via software and lights it up). It then ties the package and locker together and sends the pickup code to the customer (via email/SMS – you can just mention “notification”). For **pickup**, the customer enters the code (or scans), the system verifies and directs that locker to unlock, and logs the pickup.

- **Edge Cases:** What if no locker is available? (The system should notify that lockers are full, and perhaps queue the delivery or direct to another location.) What if a wrong code is entered? (Deny access after a few attempts perhaps).

- **Depth:** Interviewers expect a clear class design and understanding of the flow. You don't necessarily need to implement hashing of codes or a database of codes, but mention how you store the mapping from `pickupCode` to locker/package (e.g., a dictionary in `LockerManager`). Also mention any **design patterns** if relevant: for instance, Strategy could be used if there were different algorithms for choosing a locker (first-fit vs. best-fit size). But primarily, focus on correctness and completeness of the flows. Citing a real Amazon use-case can earn bonus points – e.g., mention that after some days uncollected packages are removed (though you may not need to implement that in LLD).

8. Design a Hotel Management / Hotel Booking System

Why it's asked: This is a variant of booking systems focusing on hotel room reservations. It's common in interviews to test how you model time-bound reservations and resource allocation (rooms). The question has appeared at Amazon and other companies as an OOD exercise ¹².

What to cover:

- **Scope:** Usually the design is for a single hotel (to keep it simple), but you can mention that extending to multiple hotels just means having a container of hotels. Focus on room reservations, check-in/check-out processes, and billing if relevant.

- **Core Features:**

- **Room Types & Availability:** The hotel has different types of rooms (single, double, suite, etc.). The system must manage which rooms are available for given dates.

- **Booking Process:** Customers can **reserve a room** for certain dates. On booking, that room becomes unavailable for that date range. Consider what information is needed (guest info, payment guarantee etc., though you can simplify).

- **Check-in/Check-out:** At check-in, mark the room as occupied; at check-out, free it up. Possibly generate a

bill for any additional services (this can be an extension if time permits).

- **Entities:**

- **Room**: with attributes like roomNumber, type, and a flag for availability (though availability is better derived from reservations).

- **Reservation**: which contains details of the guest, room, check-in date, check-out date, and reservation status (Booked, CheckedIn, Completed, Cancelled, etc.).

- **Guest** (or **Customer**): their info, and possibly a record of reservations made.

- Optionally, a **Hotel** class to encapsulate rooms and maybe a booking calendar structure.

- **Reservation Management**: The key challenge is avoiding double-booking. Explain how you would store reservations (e.g., in a list or calendar) and how to check if a room is free for requested dates – likely by checking existing reservations for overlap. A simple approach: iterate over reservations for that room and ensure none conflict. For an interview, that level of detail is fine (no need for fancy interval trees unless you want to impress, but not required for new grad).

- **Features/Constraints**: Mention handling cancellations (free up the room if canceled before the date). If the design includes payment, just assume the booking is guaranteed by card but you won't implement payment processing. Also mention **overbooking prevention**: the system should ensure you don't confirm more bookings than rooms available ¹³. For example, if all rooms of a certain type are booked on a date, further requests should be denied or waitlisted.

- **Depth**: Interviewers expect a discussion of class design (Room, Reservation, etc.) and how the interactions occur when making a reservation. They may ask something like, "How would you find an available room for given dates?" – be ready to walk through that logic. Also be prepared for slight extensions: e.g., "How would you extend this to support multiple hotels or online travel portals?" In an entry-level setting, a solid single-hotel design with clear classes and no obvious flaws (like double-booking) is usually sufficient ¹⁴.

9. Design a Food Ordering & Delivery System (e.g. Swiggy, Uber Eats)

Why it's asked: Food delivery combines elements of marketplace systems (restaurants, customers, delivery) and is a good test of modeling complex interactions in OOP. Variations of this are popular, especially in companies with related domains (in India, e.g., Zomato/Swiggy interviews). For Amazon or others, it's still a relevant design to test your abstraction skills.

What to cover:

- **Scope**: Focus on the core flow: browsing restaurants and menus, placing an order, and delivering it. You can simplify by assuming the delivery assignment is automatic and not delve deep into maps/GPS.

- **Core Features:**

- **Restaurant Listing**: Users can browse or search restaurants (filter by cuisine, ratings, etc.). The system should maintain a list of restaurants, each with a menu of items.

- **Placing Order**: A user selects items from a restaurant's menu, places an order. The order should capture which restaurant, which items, prices, etc., and the user's delivery address.

- **Order Assignment & Delivery**: The system assigns a delivery person (courier) to the order and tracks the delivery status (picked up, on the way, delivered). The assignment could consider proximity and availability of drivers, but at a basic level, just assume one gets assigned.

- **Ratings and Reviews**: After delivery, user can rate the order or restaurant. Restaurants might have an average rating that influences their listing.

- **Entities:**

- `Restaurant` with details and a list of `MenuItem`.
- `MenuItem` for each food item (name, price).
- `Order` containing the items ordered, quantities, total price, status (NEW, PREPARING, COMPLETED, etc.), and associated restaurant, customer, and delivery driver.
- `User` (customer) and `DeliveryDriver` (or simply `Driver`). Possibly a `RestaurantOwner` if needed.
- `Delivery` or incorporate delivery info in `Order` (like driver assigned, estimated time).
- **Important Aspects:**
 - **Multiple Views/Filters:** Users might want to view restaurants sorted by rating or popularity. Mention that the system can support different sorting – e.g., a separate service or method to get “Top rated restaurants” vs. “Popular near you”. In design terms, you might use Strategy or specify that different comparators can sort restaurants list by rating, by cuisine, etc..
 - **Notification of Order:** When an order is placed, the restaurant should be notified to start preparing. And a driver should be notified to pick it up. This implies some Observer or event mechanism in real implementations, but you can describe it as a simple method call or callback for the interview.
 - **Real-time Updates:** If discussing deeper, mention that tracking delivery in real-time (like showing driver location) is more of a system design issue; in LLD, you can restrict to order status updates (like an `updateStatus()` method on `Order`).
 - **Design Patterns:** An **Observer pattern** could be mentioned for the rating updates: e.g., if restaurants have an average rating displayed, whenever a user rates an order, the Restaurant’s rating needs updating. Also, if you considered multiple views of restaurants (by rating, by cuisine), you might be hinting at Strategy (different sorting strategies) or even the use of the **Observer** to notify a “top restaurants” list when ratings change. These are subtle and not required, but demonstrate deeper insight.
 - **Depth:** The interview expectation is to handle the major classes and interactions. For example, walk through how a user placing an order would be handled by your objects (`User` -> selects `MenuItems` -> creates `Order` -> `Order` notifies `Restaurant` and assigns `Driver`, etc.). Also consider failure scenarios like restaurant being closed or driver not found; you can mention them, though you might not implement in detail. Keep the focus on a clean separation of concerns: `Restaurant` handles menu and preparation (maybe internally marking order as accepted), the `Delivery driver` handles the logistics, and so on.

10. Design an In-Memory File System

Why it’s asked: Amazon interviewers often include questions that are essentially coding tasks wrapped in a design problem, and designing a simple file system API is one of them. This question tests both OOP design (classes for files and directories) and the ability to implement methods that manipulate a hierarchical data structure (filesystem paths). It’s also known from LeetCode (problem 588) which many candidates practice.

What to cover:

- **Core Requirement:** Implement a simplified file system supporting basic operations like:
 - `ls(path)` : List directory contents (or show file name if path is a file).
 - `mkdir(path)` : Create a new directory (and intermediate dirs if not exist).
 - `addContentToFile(filePath, content)` : Create a file if it doesn’t exist, or append to it if it does.
 - `readContentFromFile(filePath)` : Return the file’s content.
- **Entities:**
 - `Directory` and `File` classes. A `Directory` contains a map of names to child `Directory` or `File` (composite pattern approach). A `File` holds its content (as a string, for example).
 - Alternatively, a single unified `Node` class with a flag for file/dir can be used, but separating `File` and

Directory is clearer.

- The system can have a root Directory (e.g., "/").
- **Design:** Represent the file system as a tree structure. For instance, start with a root Directory that has a hashmap for subdirectories and files. Implement methods:
- `ls`: Traverse to the given path. If it's a directory, return the list of names of children (directories and files) sorted lexicographically. If it's a file, just return that file name.
- `mkdir`: Traverse the path, creating Directory nodes as needed for each segment that doesn't exist.
- `addContentToFile`: Traverse to the directory, then create a File node if needed and append content to it.
- `readContentFromFile`: Traverse to the file and return its stored content.
- **Example:** If you call `mkdir("/a/b/c")`, it should create directories `a`, then `b` inside `a`, then `c` inside `b`. If then `addContentToFile("/a/b/c/d", "hello")` is called, it creates file `d` in directory `c` with content "hello" (or appends if `d` existed).
- **Edge Cases:** Mention handling of trailing slashes, or that root "/" should list top-level dirs/files. The problem usually stipulates valid input, but it's good to be cautious (for example, ensure that you don't create a file and directory with the same name in one location).
- **Depth:** In an interview, you might actually be asked to write the code for these methods (this can be more of a coding round). But from a design perspective, focus on how you structure classes and navigate the hierarchy. A common mistake to avoid is using a flat map for paths instead of a true tree; interviewers want to see you understand hierarchical relationships. Also, emphasize efficiency: operations like `ls` and `mkdir` should operate in $O(n)$ where n is path components length (which is reasonable). Amazon specifically has asked this question, so citing that it's a known frequent question (which it is) can show you're well-prepared.

11. Design a Chess Game (Object-Oriented Design for Chess)

Why it's asked: Board game designs like Chess are popular to test how you handle complex rules and interactions in OOP. Chess requires modeling pieces with different behaviors, enforcing rules, and possibly game state evaluation. It's known to be asked in some design interviews and often appears in system design/OOD tutorials ¹⁵.

What to cover:

- **Core Components:**
- **Board:** 8x8 grid of cells. You can represent it as a class `ChessBoard` containing a 2D array of cells or perhaps a mapping from coordinates to Piece.
- **Pieces:** Class hierarchy for chess pieces – e.g., an abstract base `Piece` class and subclasses like `King`, `Queen`, `Rook`, `Bishop`, `Knight`, `Pawn`. Each piece knows its current position and can determine what moves are allowed.
- **Game / Engine:** A class to manage the game state, whose turn it is, and validate moves (including rules like check, checkmate, castling, etc., depending on how far you go). For a basic design, you may not implement all chess rules, but ensure movement and capture logic is covered.
- **Movement Logic:** Each piece has its specific movement rules (e.g., rook moves straight lines, bishop diagonals, knight in L-shape). A good design uses a **Strategy pattern** or similar to encapsulate move logic. For example, the `Piece` class could have a method `getValidMoves()` which is implemented differently by each subclass. Alternatively, you could have separate move behavior classes if you want to demonstrate patterns.
- **Turn Management:** The game alternates turns between two players (White and Black). Model players or

simply keep a boolean for whose turn it is. When a player attempts a move, the system should verify it's a valid move for that piece and that it doesn't violate any rules (e.g., can't land on a same-color piece, can't leave king in check – although check/checkmate detection can be advanced to implement fully).

- **Capturing Pieces:** If a piece lands on a square occupied by an opponent's piece, that piece is captured (removed from board). Explain how your design handles this (maybe a method on `ChessBoard` like `movePiece(from, to)` which handles updating positions and capturing).

- **Check and Checkmate:** For a complete design, mention that you'd have methods to detect check/checkmate by scanning the board for king's safety. This can be complex, but simply stating that the `Game` or `Board` class would have a method to check if the King is under attack is enough to show awareness.

- **Patterns & Design:** Many solutions mention using a **Factory pattern** to create pieces and place them at game start (since initial positions are fixed for each piece type). Singleton could be used if there was a need for a single game manager instance (though not mandatory). The key is clean class separation: Piece responsibilities vs. Board vs. Game.

- **Depth:** Interviewers might not expect you to implement every rule of chess in code, but they will look for how you structure the inheritance for pieces and handle move validation. You should at least handle basic moves and captures. A common follow-up is: "How would you extend this for other board games (like checkers)?" indicating if your design is general. Ensure your piece logic is encapsulated so that adding a new piece type or a different game's pieces is feasible. This demonstrates good OOP practice.

12. Design a Battleship Game

Why it's asked: Battleship is another classic board game design problem. It's a bit simpler than chess in terms of movement, but involves a turn-based guessing element and a different style of interaction (hidden information). Companies (even outside FAANG, like game companies or others) have asked this, and it has been reported in some interview prep communities (including hints that Google has asked it).

What to cover:

- **Game Overview:** Battleship is a two-player game where each player places ships on a grid and then players take turns calling out grid coordinates to "attack" the opponent's ships. The goal is to sink all of the opponent's ships.

- **Core Components:**

- **Board/Grid:** Typically a 10x10 grid for each player. Each cell can be water or part of a ship. You might represent it as a 2D array or a matrix class.

- **Ship:** Different ships of varying lengths (carrier, battleship, submarine, etc.). Each ship occupies a number of consecutive cells on the grid. A `Ship` class can have properties like name, size (length), and how many hits it has taken.

- **Player:** Each player has a board with their own ships, and also needs to keep track of hits/misses against the opponent. You could maintain a separate "tracking grid" for what the player knows about the opponent's board (e.g., mark hits and misses).

- **Game Master / Manager:** A class to control turn order, verify moves, and determine when the game ends (when all ships of one side are sunk). It can also handle the setup phase (placing ships, possibly randomly or by input).

- **Gameplay Logic:**

- **Setup:** Each player places their ships on their board (could be manual input or predetermined for the design's sake). Ensure no overlapping ships and all on board.

- **Turns:** On a turn, a player calls a coordinate (e.g., "A5"). The system checks the opponent's board at that coordinate to see if it's a hit or miss. It then updates state: mark a hit on that ship (and maybe reduce ship's

health), or mark a miss. Feedback is given ("hit" or "miss", and if a ship is sunk, announce it).

- **Win Condition:** If all ships of a player are sunk, the opponent wins.
- **Key Methods:** For example, `Player.takeTurn(coord)` which calls into a method in `Game` like `attack(player, x, y)`. The `Game` would then update the boards appropriately. You might have `Board.placeShip(ship, startCoord, orientation)` for setup, and `Board.receiveAttack(x,y)` to handle being attacked (return hit/miss and update ship status if hit).
- **Design Considerations:** Battleship can introduce the concept of **hidden information** – player A should not know the positions of player B's ships. In design, you can enforce this by not exposing opponent's board details and only providing results of attacks. Since this is an OO design, you could imagine that each `Player` has a method to attack the opponent, which internally calls the opponent board's method; the opponent's board can be encapsulated.
- **Depth:** Emphasize a clean separation: the `Ship` knows its coordinates (or at least how many hits left), the `Board` knows where ships are and handles hits, the `Game` or `Judge` manages turns and victory. If pressed for patterns, you might say this is an example where no complex patterns are strictly needed, but you are using encapsulation and perhaps an Observer if you wanted to notify players of events (hit/miss) – though usually a simple function call suffices. Since battleship has relatively simpler rules, interviewers might expect a bit more detail on how you'd implement certain methods. Be ready for follow-ups like "How to check if a ship is sunk (all its cells hit)?" – you'd answer that each ship tracks hits or the board checks all coordinates of that ship.

13. Design a Ride-Sharing Service (e.g. Uber/Lyft) – Low-Level

Why it's asked: Designing a ride-sharing service at a low level tests your ability to abstract a real-world system with moving parts (drivers, riders, rides) without getting lost in the large scale. It's less common for entry-level than a pure OOP exercise, but aspects of it can be asked to see how you handle dynamic matching and state updates in OOP ¹⁶.

What to cover:

- Core Entities:

- `Rider` (or Passenger): who can request rides. Has details like name, payment info, and possibly current location.
- `Driver`: has a vehicle, current location, status (available or on a trip).
- `Ride` (or `Trip`): representing an ongoing or completed ride, including rider, driver, origin, destination, fare, status (requested, ongoing, completed).
- Optionally, `Vehicle` as a separate class if you want to attach attributes (type of car, plate number, etc.).

- Main Flow:

- **Requesting a Ride:** A Rider requests a ride specifying a destination (and maybe current location if system doesn't automatically get it). The system should **match** the rider with an available driver. This matching could be based on nearest driver or other criteria ¹⁷. In an LLD context, simply say "find the closest available driver to the rider's location."
- **Driver Assignment:** Once a driver is assigned, a `Ride` object is created linking the rider and driver. The driver's status changes to "en route/picked up".
- **Ride Execution:** The driver picks up the rider and eventually drops them at the destination. The system might track the route or at least know when the ride is finished.
- **Completion:** Mark the ride complete, driver becomes available again, fare calculated (could be a simple function of distance/time which you might not implement fully). Possibly allow rating the ride.
- **Design Challenges:** The key is matching algorithm and state management: how do you keep track of

available drivers? In a simple design, you might maintain a list or map of available drivers that you search through (which is fine to mention). Also, location could be an attribute (latitude/longitude) updated periodically – but you can simplify by not diving into tracking.

- **Interactions:** The `RideSharingService` (or `Dispatcher`) class can handle ride requests. For example, `requestRide(rider, fromLocation, toLocation)` method: it finds a driver (perhaps by scanning a list of drivers with status available, computing distance), assigns one, and returns a `Ride`. The driver might have a method to accept the ride (or we assume auto-accept for simplicity).

- **Extensibility:** Mention that many real-world aspects (surge pricing, cancellations, multi-rider pooling) exist but are out of scope. It's good to show awareness but clarify the simple case first.

- **Depth:** Interviewers expect a solid understanding that this is essentially a many-to-many matching problem simplified to one rider to one driver at a time. Focus on the OOP: classes and their methods. For instance, how does a `Driver` get updated location? Maybe a method `updateLocation()` that the app would call – but you can abstract that. If asked how to optimize finding nearest driver, you might say use spatial indexing or quad-trees, but that veers into system design. At LLD level, “just iterate over drivers and pick the closest for now” is acceptable, as long as your design cleanly separates concerns (e.g., the matching logic in one place). Also highlight thread-safety concerns if multiple requests come (maybe a simple lock around driver assignment to avoid two riders getting the same driver).

14. Design Splitwise (Bill Splitting Application)

Why it's asked: Splitwise or a bill splitting app is a common machine coding question, especially in fintech or social finance contexts. It requires managing users, expenses, and balances – showcasing how you'd use data structures to track debts and credits between multiple people. Amazon has occasionally asked similar questions (like splitting payments) as have other companies, because it involves algorithmic thinking plus OOP design.

What to cover:

- **Basic Scenario:** Friends share expenses and the app should tell who owes whom how much. For example, if Alice and Bob split a \$100 dinner, each should pay \$50, so if Alice paid, Bob owes Alice \$50. The system should maintain running balances for each pair of users.

- **Core Features:**

- **Add Expense:** When an expense is added, involving one or more people, update the balances. Expenses can be split equally or in specified ratios or percentages (depending on complexity – basic equal split is fine to start).

- **Show Balances:** For a given user (or all users), show how much they owe or are owed in total, possibly broken down by friend. E.g., “Alice owes Bob \$50, Alice is owed \$30 by Charlie,” etc.

- **Settle Balance:** An optional feature is to record a payment between users to settle debts (like if Bob pays Alice \$50 outside the app, we record that to clear the debt).

- **Entities:**

- `User`: basic info and perhaps a balance sheet (could just be an ID and name, with balances stored elsewhere).

- `Expense`: record of an expense with amount, payer (who paid), participants (who should share it), and how the split is determined (equal or percentages, etc.).

- You might not need a complex class for balance; a data structure like a map of (User -> amount) to represent each user's net balance with respect to others can suffice. However, some designs introduce a `Transaction` or `Balance` class to encapsulate an IOU.

- **Tracking Balances:** One straightforward approach: maintain a **map of maps** structure,

`balances[payer][payee] = amount` meaning payer is owed "amount" by payee. Or vice versa, decide a consistent meaning (like a positive number means A owes B). Many solutions keep a matrix or map for debts. When a new expense is added, you update these balances accordingly. For instance, if expense of \$100 is split between A, B (A paid), then update record so that B owes A \$50.

- **Example:** If A pays \$120 for A, B, C equally, then B owes A \$40 and C owes A \$40 (assuming A, B, C are distinct). If later B pays an expense that results in A owing B, you offset against any existing balances between A and B. The system should ideally simplify debts: rather than A owes B \$30 and B owes A \$20, that simplifies to A owes B \$10. This can be done by netting out amounts when updating balances.

- **Depth:** An interviewer will look if you handle the data structures correctly and understand the arithmetic. They might ask you to actually implement an `addExpense` method and a `showBalances()` method. Make sure to cover both equal splits and mention how you'd extend to uneven splits (maybe provide an example). Also mention how the data model would work for group splits with many people. In terms of design, it's more about correctness and less about patterns, but you could mention using Factory for creating different types of splits (equal, exact amounts, percentage) in a more advanced solution. The key is to avoid confusion in representing who owes whom; be very clear with your chosen representation and be ready to walk through an example to demonstrate it.

15. Design an ATM Machine Software

Why it's asked: Designing an ATM involves thinking about hardware-software interaction and handling of states (card inserted, PIN entry, transaction selection, etc.). It's a common LLD question that tests knowledge of OOP for modeling real-world devices and processes ¹⁸. Companies like Amazon or large banks might ask this to see how you handle workflows in design.

What to cover:

- **ATM Workflow:** Break down the ATM usage steps – card insertion, PIN validation, menu selection, transaction processing (cash withdrawal, balance inquiry, etc.), and card eject. This is inherently a state-driven process.

- Core Features:

- **Authentication:** The ATM should authenticate the user by reading card info and verifying PIN (likely by communicating with a bank system). For design, you can assume if PIN matches the card's account, authentication succeeds ¹⁹.

- **Transactions:** Support operations like **Withdrawal, Deposit, Balance Check, Transfer**, etc. At least handle Withdrawal and Balance Check for simplicity.

- **Cash Dispensing:** If withdrawal is requested, ensure the ATM has enough cash and the account has sufficient funds, then dispense cash and deduct from account.

- **Hardware Interaction:** Consider components like Card reader, Screen, Keypad, Cash Dispenser, Cash Receiver (for deposits), Printer (for receipts). Each can be a class or at least conceptualized.

- Entities:

- `ATM` class as the central machine controller. Contains components (as above) and perhaps the state (like current session info).

- `Card` or `BankAccount` representation: The card might contain an account number; you could have a `BankService` or `AccountManager` to which the ATM talks to fetch account balance or apply debits/credits ²⁰. In an OOP design, you might simulate this with an `Account` object for the user.

- `Transaction` class hierarchy: Could have abstract `Transaction` and subclasses `WithdrawalTransaction`, `DepositTransaction`, etc., each with a `execute()` method. This is a nice way to apply OOP for different operations.

- **UserSession**: optional, to track the authenticated user's info (account, remaining attempts for PIN, etc.) during one ATM session.
- **State Design**: You can mention using a **State pattern** for ATM states: e.g., Idle state (waiting for card), Auth state (waiting for PIN), Selection state (menu), Transaction Processing state, etc. Each state defines allowed actions. This can ensure, for instance, you can't dispense cash before authentication. Using state pattern is a strong design choice here (though at entry level just explaining states without formal pattern is fine).
- **Security Considerations**: Mention that after a few wrong PIN attempts the ATM should retain the card or disable further tries (this shows attention to real-world detail). Also mention timeouts (if user is idle too long, session cancels).
- **Depth**: Focus on class interactions: ATM uses CardReader to read card, calls bank service to validate PIN, then shows menu. When user selects withdraw, ATM checks its cash bins (e.g., **CashDispenser** class with methods to dispense a certain amount and know how many bills remain). The ATM updates the account balance via the bank service. After transaction, the card is ejected. Interviewers will likely ask something like, "What happens if the ATM runs out of cash?" (Your design should allow checking the cash level and refusing transactions if empty). Also be prepared for concurrency aspects – e.g., ATM serving one user at a time, so concurrency is mainly with the central bank system updating accounts. By covering the main flow thoroughly and mentioning a few edge cases, you'll demonstrate a solid understanding ¹⁸.

16. Design a Pizza Ordering/Billing System (Pizza Shop OOD)

Why it's asked: This question is often used to assess knowledge of design patterns (specifically **Builder** or **Decorator** patterns) and OOP fundamentals. Amazon interviewers have asked candidates to design a pizza ordering system focusing on how to add toppings and calculate cost. It's a relatively constrained domain that lets you showcase class design for items with optional components.

What to cover:

- **Use Case**: A customer can build a pizza, choosing size and adding toppings, and the system should calculate the final price. Optionally, consider multiple pizza orders, but often the emphasis is on modeling a single pizza's composition and cost.
- **Core Features**:
 - **Pizza Composition**: Pizzas have a base (like regular or thin crust), a size (Small, Medium, Large), and various toppings (cheese, pepperoni, veggies, etc.). Each of these choices affects price.
 - **Cost Calculation**: The system should be able to compute the price of a pizza based on its base price plus the cost of each topping. If there are size-based price differences, include that.
 - Possibly, **Pizza Customization**: ensure that one can easily add or remove toppings from a pizza object, and maybe get a description of it (to print receipt).
- **Entities**:
 - **Pizza**: base class or interface that defines methods like `getCost()` and `getDescription()`.
 - Could have concrete classes like **MargheritaPizza**, etc., but a more flexible approach is to have a basic **Pizza** class and then apply toppings dynamically (using Decorator pattern). Alternatively, use the Builder pattern to construct a pizza with various options.
 - **Topping**: Each topping could be a class (if using Decorator, each topping is a decorator wrapping a Pizza). E.g., **CheeseTopping** that adds cost and description of cheese.
 - If not using Decorator, you might simply have Pizza contain a list of topping objects or an enum, and have logic to sum up cost. But interviewers often expect to hear about **Decorator vs Builder** here.
- **Design Patterns**:

- **Builder Pattern:** Often suggested when constructing a complex object like Pizza with many options. You can describe a `PizzaBuilder` that lets you set size, base, and add toppings with chained methods, finally producing a Pizza object. This is useful to avoid a telescoping constructor if Pizza had many parameters.
- **Decorator Pattern:** Allows adding toppings as wrappers around a Pizza object dynamically. For example, a `PlainPizza` starts with base cost, then you wrap it: `pizza = new CheeseDecorator(pizza)` adds cheese cost, then `pizza = new PepperoniDecorator(pizza)` adds pepperoni, etc. The final object's `getCost()` sums all decorators' costs. This pattern is a classic solution for the pizza topping problem; it treats toppings uniformly and is very extensible.
- You can mention either or both patterns. In some discussions, Builder is favored for assembling the pizza, and Decorator for calculating cost dynamically. One could even combine: use Builder to choose toppings, then internally the Pizza object uses decorators.
- **Cost Calculation:** Give a simple example – e.g., “A medium pizza base costs \$8, each veggie topping +\$1, each meat topping +\$2. So if I build a medium pizza with mushrooms and pepperoni, the cost = 8 + 1 (mushroom) + 2 (pepperoni) = \$11.” This proves your design works for computing price.
- **Depth:** Interviewers will gauge your understanding of how to design for extension. If they ask “How would you add a new topping in your system?”, the ideal answer if using Decorator: “Just create a new decorator class for that topping, implementing the same topping interface, and it will seamlessly integrate.” If using Builder/lists: “Add it to the list of toppings and define its cost, the code auto sums it.” Also consider mention of a possible `enum ToppingType` to map topping names to costs (a simpler approach). But since the question specifically alludes to “which design pattern is best for this”, having a pattern-based answer ready (Decorator or Builder) is wise. In an Amazon context, one candidate noted the builder pattern with method chaining as a good approach (like `.addTopping("Olives").addTopping("Tomatoes")` returning the builder for chaining).

17. Design a Notification System (Observer Pattern Use-Case)

Why it's asked: A notification system design checks your ability to handle asynchronous events and multiple delivery channels in OOP. It's often used to see if you know the **Observer pattern** and can separate concerns (e.g., content vs. delivery). It might come up in interviews where the role involves designing user-facing features (like alerts, emails) or just as a theoretical exercise ²¹.

What to cover:

- **Purpose:** The system sends notifications (e.g., order updates, alerts, reminders) to users through different channels (email, SMS, push notifications, etc.). Design a flexible system to support this.
- **Core Components:**
 - **Notification:** an object representing the message content and metadata (type, timestamp, etc.). For example, a notification could be “Your order has shipped!” or “Password changed”.
 - **User:** who will receive notifications. A user might have preferences for channels (maybe user A wants email and SMS, user B only push).
 - **Channels (Delivery Methods):** such as `EmailService`, `SMSService`, `PushService`. Each knows how to deliver a notification via that medium (e.g., via SMTP, SMS gateway, etc., though you won't implement those in detail).
 - **Notification Service/Manager:** the core class that takes in new notifications and dispatches them to the appropriate channels for each user. This is where the Observer pattern can come in – each user or channel can be an observer subscribed to certain event types.
- **Design Patterns:**
 - **Observer Pattern:** A classic approach is to have observers for users or channels. For instance, the system

could maintain a list of observers (like an EmailObserver, SMSObserver) that listen for notifications and handle sending ²². Or observers per user that know that user's preferred channels. When a new Notification event is published, all relevant observers are notified to actually deliver it. This decouples the notification generation from how it's delivered.

- **Strategy Pattern:** Could be used for formatting or prioritizing – e.g., different strategies to pick channel based on urgency. But this is less common than Observer here.

- **Flow:** Suppose an order status changes in an e-commerce system, it creates an "OrderShipped" notification. The NotificationService receives this and looks up which users need to be notified and via what channels. It then either calls the respective channel services or notifies observers representing those channels. Each channel service formats the message suitably (email template vs. SMS text) and sends it out ²¹.

- **Reliability:** Mention ensuring delivery – e.g., if email fails, maybe retry or fall back to SMS. This might be advanced, but at least mention reliable delivery (like "we might queue notifications and mark as sent or failed").

- **Depth:** Focus on class design: a Notification class, separate channel classes, and the manager or dispatcher. Show you can easily add a new channel (e.g., add SlackNotificationService for Slack alerts) without changing the core – this is key for extensibility. Interviewers might ask "how to add a new notification type or channel?" The answer should be: create a new subclass or observer, etc., rather than modifying a lot of existing code (demonstrating OCP from SOLID). Since this is conceptual, code may not be required, but they may nudge you to describe how Observer works in your design (publish-subscribe model).

18. Design a Customer Support Ticketing System

Why it's asked: This problem (sometimes framed as "issue tracking system" or similar) evaluates how you design workflows and apply patterns like strategy for assignment. Companies that have customer service products (or any internal tool development) might ask this to see how you handle stateful processes and different roles (customer vs agent). It was noted in an interview for a payments company and PhonePe etc..

What to cover:

- **Scenario:** Customers can create support tickets (issues) which customer service agents then pick up and resolve. The system must manage ticket creation, assignment to agents, and status updates.

- **Core Features:**

- **Ticket Lifecycle:** A ticket can be created (New), assigned to an agent (Open/In-Progress), marked Resolved/Closed when done. Possibly states like Pending Customer Response, etc., but keep it simple unless asked.

- **Agent Assignment:** When a ticket is created, the system should assign it to an appropriate support agent. How to choose an agent can vary (least loaded agent, or based on expertise tags, etc.).

- **Ticket Tracking:** Users should be able to see their ticket status, and agents should see tickets assigned to them. Agents may update ticket status or reassign if needed.

- **Entities:**

- **Ticket**: with fields like id, description, customer, assignedAgent (could be null if unassigned), status, priority, tags (if categorizing issues), timestamp, etc.

- **User**: specifically differentiate **Customer** (who creates tickets) and **Agent** (who resolves). Possibly use inheritance or roles. Agents have properties like expertise area, current workload (number of open tickets).

- **TicketingSystem** or **TicketManager**: to create tickets and handle assignments.

- Optionally **AssignmentStrategy**: an interface with a method to select the best agent for a ticket;

implement multiple strategies (round-robin, least load, skill-based, etc.). This is where **Strategy pattern** is useful.

- **Assignment Logic:** Mention that different companies might assign tickets differently. For flexibility, use Strategy for assignment. For example, a `LeastLoadedStrategy` finds the agent with fewest open tickets; a `SkillBasedStrategy` might match ticket tags to agent specialties. The system can choose one or the other (perhaps configured by admin).

- **Observer Usage:** If the assignment strategy needs to react to changes (like an agent closing a ticket means their load changed), you could use Observer to notify the strategy or a central manager of state changes. However, simply recomputing on each assignment is fine too. Observer could also be used to notify agents (e.g., their interface updates when a new ticket is assigned to them).

- **Ticket Updates:** Outline how an agent updates a ticket (e.g., adding a comment or resolution, changing status). The system should record these events (could have a log or just change state). If a ticket is resolved, maybe it triggers an email to customer – tie in with the Notification system design if relevant, but not required.

- **Depth:** Interviewers will look for good class decomposition and application of patterns. Emphasize the Strategy pattern for assignment as a highlight. Also, ensure you handle basic integrity: an agent shouldn't be assigned more tickets than they can handle (unless that's part of strategy). If the question was specifically low-level (machine coding), you might be asked to implement a simple assignment logic in code. But as design, focusing on flexibility and clarity is key. For instance, mention how to add a new agent assignment criterion easily (just create a new strategy implementation). This demonstrates forward-thinking design.

19. Design an Inventory and Order Management System

Why it's asked: This is about designing a system to manage stock levels and orders, commonly encountered in e-commerce or warehouse management. It tests understanding of concurrency (if multiple orders come in) and consistent state updates. Amazon, dealing with fulfillment, values such design skills. In interviews, a simplified version might be asked as a machine coding test or LLD question (e.g., manage inventory for sellers).

What to cover:

- **Scope:** Imagine a platform where sellers list products with quantities. Customers place orders which reduce inventory. The system should provide functions to add inventory, place orders, and query inventory levels.

- Core Features:

- **Inventory Management:** Ability to increase stock or add new products (e.g., a seller receives new stock for a product).

- **Order Placement:** Given a product and quantity, create an order if stock is available, and decrement the inventory count. If stock is insufficient, prevent the order or perhaps put it on backorder (depending on design, but simplest is to reject if not enough inventory).

- **Queries:** Fetch current inventory for a specific product, or list all products below a certain stock threshold (to trigger restock, perhaps). Also, find which sellers can fulfill an order of a certain quantity (if multiple sellers).

- Entities:

- `Product` (or `Item`): has an ID, name, description, etc. If multiple sellers, include a SellerID or design a separate entity for Seller's inventory.

- `InventoryRecord`: could map a Product (and possibly Seller) to the quantity on hand. You might

integrate this into Product class or have a separate class managing stock counts.

- **Order**: with details like orderID, product, quantity ordered, status (Created, Fulfilled, Cancelled), timestamp, and reference to customer if needed (though for pure inventory management, you can skip customer details).
- Possibly a **Warehouse** or **Seller** class if multiple sources of inventory.
- **Operations:**
 - **addInventory(productId, quantity)**: Increase available count for that product. If product doesn't exist in system, create a new record (for a new product).
 - **createOrder(productId, quantity)**: Check current inventory. If `available >= quantity`, allow the order – decrement inventory and return success (and perhaps an Order object/ID). If not enough, either throw an error or create an order with status “Pending” (but that complicates things). Usually, we'd reject or waitlist.
 - **getAvailableInventory(productId)**: returns the current count. (This could also be a property accessible directly, but a method is safer for abstraction).
 - If multiple sellers: **findSellersForProduct(productId, quantityNeeded)**: returns list of sellers who have at least that quantity in stock.
- **Concurrency**: If this comes up, mention that if two orders for the same product come simultaneously, you'd need to synchronize the inventory update (to avoid selling more than you have). In code, that could mean using transactions or locking around the decrement. Since the question is often a machine coding round, using thread-safe structures or simple locks might be expected. The prompt in one source explicitly noted multi-threading importance for this problem, so it's good to mention using appropriate locking or atomic operations to handle concurrent inventory updates.
- **Depth**: Keep the design straightforward. The main complexity is ensuring consistency (inventory never goes negative, orders are fulfilled correctly). If asked to support features like “reserved stock” (for an order in process) or “restock alerts,” you can outline how to extend the design (maybe an Observer to notify when inventory drops below threshold). But base requirements are adding and consuming stock safely. Emphasize reliability – e.g., if an order creation fails midway, make sure any partial deduction is rolled back (though in a simple in-memory design, you'd do checks then deduct in one go). Also, demonstrate clarity on the relationship: one product can have many orders, inventory is basically one number per product (or per product per seller). Use of design patterns might not be necessary here, but you could mention Singleton for the Inventory manager if only one centralized store exists, or Factory for creating orders, etc., if you want to showcase pattern knowledge.

20. Design a Chat Application (Simple Messaging)

Why it's asked: Designing a chat app checks how you manage real-time communication and data model for messages, users, and chat rooms. While a full chat system has heavy networking aspects (which might be more HLD), an LLD can focus on the relationships between users, messages, and conversations. It's a good test of modeling and sometimes appears in interviews (especially for roles dealing with messaging or social features) ²³.

What to cover:

- **Scope**: Limit to a basic chat feature: users can send messages to each other (1-1 chat) or possibly in chat rooms (group chat), and messages should be stored/retrievable. Ignore the low-level networking (assume sending via server is instantaneous).
- **Core Features**:
 - **User-to-User Chat**: Two users can start a conversation and send text messages back and forth. The

system should track the messages in order and who sent each.

- **Group Chat (optional):** Multiple users in a chat room/channel. Messages sent are seen by all in the group. This adds complexity of membership management (users can join/leave groups). If time is short, you might skip groups, but it's a nice extension.

- **Message Delivery:** Ensure that if user A sends a message, it can be delivered to user B. Possibly consider online/offline – e.g., if B is offline, the message should be stored and delivered when they come online.

- **Persistence:** Not always asked, but mention that messages would be saved so they can be loaded when a user opens the app later (database in real world; for LLD just say stored in memory structure or a list).

- **Entities:**

- **User** : has userID, name, maybe an online status. Also may have a list of conversations they are part of.

- **Message** : contains sender, timestamp, content, maybe a unique ID. Could also have a status (sent, delivered, read if you want to track read receipts).

- **Conversation** or **Chat** : an object representing a chat session. For one-on-one chat, it might just have two participants; for group, many participants. It contains a list of Messages. In one-on-one case, you could identify conversation by the pair of userIDs, but having a class is cleaner.

- If supporting group chat: **ChatGroup** extending Conversation, with methods to add/remove members, and a list of members.

- **Message Flow:** Describe how sending works: e.g., `sendMessage(sender, receiver, text)` – the system finds or creates the Conversation between sender and receiver, creates a Message object, adds it to the conversation's message list, and notifies the receiver. For notification, you might call a method on the **User** or a **NotificationService** (tie-in to earlier design) to deliver the message if online ²⁴.

- **Real-time Delivery:** Could mention use of Observer pattern (each user's client is observing new messages in their conversations). But since it's LLD, just assume the system can call a method like `receiver.receiveMessage(msg)` if they're online. If offline, perhaps store it in conversation and mark as unread.

- **Additional Considerations:** Authentication (users logging in) is usually out of scope unless specifically mentioned. Focus on the messaging itself. For read receipts, you could have a method to mark messages as read and maybe notify the sender (this complicates a bit, only mention if asked).

- **Depth:** Interviewers expect class definitions and relationships. For example, explain that a **Conversation** has participants and messages, a **User** can be in multiple conversations. Maybe discuss how you'd find all messages between two users (like load the Conversation object). Also consider scalability in small: if asked "how would this scale to many users?", you might mention the need for servers, but that's HLD. In LLD, they might be satisfied with how you manage references (e.g., ensure no memory leaks by cleaning up old messages or something). One crucial thing: privacy – ensure that only participants of a conversation can see its messages (the design naturally ensures that if you encapsulate messages within Conversation and only allow access if user is part of it).

21. Design a Social Media Feed (Mini Twitter/Facebook feed)

Why it's asked: This tests modeling of a social network's basic components: users, posts, comments, etc., and possibly the generation of a "feed". At an LLD level, it focuses on class structure for posts and relationships rather than the huge scaling challenges. It's relevant for companies like Meta and sometimes asked in a simplified form for new grads to see understanding of associations and maybe some design patterns for news feed generation ²⁵.

What to cover:

- **Core Features:**

- **Posting:** Users can create posts (text, image, etc.).
- **Feed:** Users can follow other users; a user's feed is a list of recent posts made by people they follow (plus their own posts, typically).
- **Interactions:** Liking or commenting on posts. Focus on these as they involve relationships between entities (user-post like, user-post comment).
- **Profile/Timeline:** Possibly mention that each user also has their own timeline of posts they've made.
- **Entities:**
 - **User**: attributes like userID, name, etc., and collections such as a list of **followers** and **following** (who they follow).
 - **Post**: contains content, timestamp, author (User), and possibly a list of comments and likes.
 - **Comment**: a smaller entity with content, author, timestamp, and maybe attached to a Post (or could allow nested comments, but that gets complex – skip nesting unless asked).
 - **Like**: could be as simple as a record that a user liked a post. You can represent likes as just a count on Post plus maybe a set of users who liked (for preventing multiple likes by same user).
- **Feed Generation:** In a real system this is complex (pull vs push model, ranking algorithms), but for LLD, you can simplify: The feed for a user is composed by aggregating posts from all users they follow, sorted by time. So you might have a method like `getFeed(user)` that goes through each followed user's posts and merges them sorted. This is not efficient at scale, but demonstrates the idea. If you want to incorporate some pattern, you could mention an **Observer**: each user's feed could be an observer to the people they follow, getting notified when a new post occurs (then adding it to feed) ²⁶. This decouples things a bit.
- **Privacy and Access:** Assume a simple model where all posts by a user are visible to their followers. No need to implement complex privacy settings unless specifically asked.
- **Interactions Detail:**
 - When a user **likes** a post: call `post.addLike(user)`. Internally, it could increment a like count and maybe log the user ID in a set. The design should ensure each user's like is counted once (so using a Set of userIDs helps).
 - When a user **comments**: create a **Comment** object with the text and author, and add it to the `post.comments` list.
 - You might mention notifications (e.g., notify post author that someone liked or commented) – which ties into the Notification System design. But only mention if time permits as an integration point.
- **Depth:** The classes and their relationships are key: **User** has relationships to **Post** (one-to-many for their own posts), **Post** has many Comments and Likes (one-to-many relationships), and the follow relationship is user-to-user. Be prepared to diagram or describe these clearly. Interviewers may ask, "How would you design the follow system?" – answer: maintain a list (or set) of follower IDs and following IDs for each User, possibly providing methods `follow(otherUser)` and `unfollow(otherUser)` that update both users' lists. Also, consider the size of these lists and maybe mention they'd likely be stored in a database for real, but in OOP it's fine as in-memory collection. The goal is to see if you can capture a simplified social network in an OO way.

22. Design an Airline Reservation System

Why it's asked: This is similar to hotel booking but with additional complexity like seats on flights. It tests ability to handle scheduling and reservations with unique constraints (e.g., each flight has a limited number of seats). It's a classic design question sometimes used in interviews for travel domain companies or as a broad design exercise ²⁷.

What to cover:

- **Scope:** Focus on booking seats on flights. That includes flights scheduled (with date/time and aircraft), and passengers booking available seats. Payment can be left out or assumed handled elsewhere.

- **Core Features:**

- **Flight Scheduling:** Flights have a flight number, origin, destination, departure time, and a set of seats (with seat numbers). Each flight belongs to an `Airline` possibly, but you can simplify by treating flights globally.

- **Seat Inventory:** Each flight's available seats must be tracked. When a booking is made, a specific seat (or seats) are reserved for the passenger.

- **Reservation (Booking):** Passengers can book a seat on a flight. The system should create a reservation record and mark that seat as booked so no one else can take it.

- **Cancellation:** If a reservation is canceled, the seat becomes free again.

- **Entities:**

- `Flight` : has flightNo, origin, destination, datetime, and `seats` – which could be a collection of `Seat` objects. Each `Seat` might have a seat number and a flag if it's booked, or you can manage bookings separately. Also possibly capacity, but seat list implies capacity.

- `Seat` : could include seat class (economy/business), etc., but at least seat identifier and availability.

- `Passenger` or `Customer` : the person booking. Contains personal info (name, maybe ID/passport).

- `Reservation` (or `Ticket`) : linking a Passenger to a Flight (and specific seat). Fields: reservation ID, flight, passenger, seat number, status (confirmed or canceled). Optionally price or payment status.

- **Booking Process:**

- Find flights for a route and date (search function). Then for a chosen flight, choose an available seat. You could have a method like `Flight.getAvailableSeats()`.

- Create a Reservation: ensure the seat is still available (to avoid race conditions ideally), then assign it to the passenger and mark it booked. Save the reservation record.

- If payment is in scope, you'd integrate that, but you can assume payment succeeds for design purposes.

- **Seat Assignment:** Mention if needed how seat classes might be handled (maybe separate seat lists for economy/business or an attribute in `Seat`). Also, frequent flyer or loyalty could be an extension but not needed basically.

- **Avoiding Conflicts:** Ensure that your design doesn't allow double booking of the same seat. If coding, you'd put a lock on the flight's seat data during booking. In design, emphasize that the `Flight` can verify seat availability and update atomically (maybe by a synchronized method).

- **Depth:** The interviewer will likely check if you considered things like multiple flights (so you might have an `AirlineSystem` or so where all flights are stored and can be searched). But at LLD, focusing on the flight and reservation relationship is key. You should demonstrate how a reservation is created by walking through the classes: e.g., a `BookingService` could have method `bookSeat(flightId, passenger, seatNo)` that finds the `Flight`, checks seat, creates `Reservation`. Also mention what happens if the flight is full (you'd throw an exception or return a failure). Citing real-world constraints like not allowing booking after flight departure, or needing check-in, can show extra insight, but only if you have time after covering basics. The essential part is mapping out flights, seats, and reservations clearly ²⁸ ²⁹.

23. Design a Meeting Room Scheduler

Why it's asked: This is a calendar-like problem focusing on booking time slots for resources (meeting rooms), which tests handling of time overlaps and scheduling conflicts. It can be asked to see how you model time-based availability and possibly use algorithms to check conflicts. It's simpler than full calendar design but still rich enough for discussion ³⁰.

What to cover:

- **Scenario:** There are multiple meeting rooms in an office. Employees can book a room for a meeting at a certain time (say start and end time). No two meetings should overlap in the same room. The system should allow checking availability and booking if free. Also consider attending personnel availability if included.

- **Core Features:**

- **Room Inventory:** Keep a list of meeting rooms (with maybe names or numbers, capacities, etc.).

- **Booking a Room:** Given a meeting's start time, end time, and required attendees (or at least the organizer), find an available room and book it. Or if a specific room is requested, check if it's free then book.

- **Checking Availability:** Ability to check which rooms are free at a given time or to list the next available slot for a room.

- **Attendee Availability (optional):** Some formulations include checking that all required attendees are also free at that time (like no double-booking a person). This significantly ups the complexity as it requires tracking each person's calendar. If time allows, you can mention it, but if not, focus on room availability alone (which is what many expect for LLD). ³¹

- **Entities:**

- **MeetingRoom**: with an ID/name, maybe capacity or location info. It could have a schedule (list of bookings or time slots reserved).

- **Meeting** (or **Reservation**): includes meeting room, startTime, endTime, organizer, list of participants.

- **User** (or **Employee**): who can schedule or attend meetings. If considering attendee availability, each User would have a calendar (list of meetings they're attending).

- Possibly a **Scheduler** or **CalendarService** class to manage the overall scheduling logic.

- **Scheduling Logic:**

- To **book** a meeting: you'd typically select a room first. For a given room, you need to check the list of existing meetings for any conflict with the requested time. That requires checking if `(newMeeting.start < existing.end && newMeeting.end > existing.start)` for any existing meeting (i.e., time overlap). If none conflict, you can save the new meeting for that room.

- If you need to *find* a free room for given time: iterate through rooms until you find one that has no conflict at that time. This is fine for small scale.

- **Double Booking Prevention:** The above logic ensures a room isn't double booked. If also avoiding double-booking a person, you must similarly check that none of the participants (especially the organizer) have another meeting overlapping that time (i.e., check their personal calendars). This is essentially the meeting room problem plus a typical calendar clash check.

- **Features:**

- **No time constraints:** The question snippet mentioned "Meeting room can be allocated at any time in a day... any duration" ³¹ – meaning no fixed slots, which implies you must handle arbitrary start/end times.

- **Listing availability:** Possibly provide a method to get all free time slots for a given room or all rooms on a day. That involves subtracting booked intervals from the day's timeline – could mention but not necessary if short on time.

- **Booking Confirmation:** Once booked, presumably participants should be notified (out of scope, but you can tie to Notification system).

- **Depth:** Focus on demonstrating how to represent bookings and check conflicts. Perhaps mention using a sorted data structure (like storing meetings sorted by start time for each room) to make conflict checking easier (just check the previous and next meetings relative to the requested time). But at a basic level, iterating is fine. Also, consider edge cases: meetings that end exactly when another begins should be allowed (no overlap if one ends at 10:00 and another starts at 10:00). Ensure your logic accounts for inclusive/exclusive boundaries properly. This problem is about correctness in scheduling, so interviewers may give scenarios to test your design: "If someone tries to book Room A from 9-11 when there's already a

10-12 meeting, what happens?" – answer: you detect conflict and refuse the booking or suggest another room. If you included attendee availability, they may ask how you'd store and check that – you'd say each User has their own list of meetings and you'd run a similar conflict check for each user's schedule during the booking operation. Covering that shows a comprehensive solution.

24. Design an Employee Management (Hierarchy) System

Why it's asked: Designing an org chart or employee hierarchy tests how you use inheritance/composition to model "is-a" vs "has-a" relationships (like Manager is an Employee who has reports). Amazon and others have asked new grads to design a simple org hierarchy and implement methods to query it. It also touches on tree traversals for things like listing reports.

What to cover:

- **Scenario:** Model an organization's employee structure. Typically, each employee has a manager (except the CEO) and may have direct reports (subordinates). You might be asked to implement functions like `getDirectReports(manager)` or `getAllReports` up to certain levels, grouping by some criteria. The example from a real interview involved grouping reports by department and experience level, so the design can be extended to handle such attributes.

- Core Features:

- **Hierarchy Structure:** Represent the management tree. Each employee should know who they manage and who their manager is (or at least one direction of linkage, usually manager -> list of direct reports).

- **Employee Information:** Employees have attributes like name, role, department, years of experience, etc. Those were explicitly mentioned in the example (name, age, department, experience).

- **Query Functions:** - Get all direct and indirect reports of a given employee (i.e., the subtree under that employee).

- Possibly filter or group those reports by certain attributes (e.g., group by department, or by experience bracket).

- Perhaps find the "lowest common manager" of two employees (a classic problem, though not in the provided example but sometimes asked).

- Entities:

- `Employee`: base class with fields for personal info and perhaps a reference to their manager. A boolean or enum for role (like if needed to mark someone as Manager vs Individual Contributor) but any employee can potentially have reports.

- Possibly subclasses like `Manager` that extend `Employee` to add managerial functions, but since any employee could have 0 or many reports, a single class might suffice with a list of `directReports`.

- `Organization` or `OrgChart`: a class to hold all employees and provide the methods to query the hierarchy. This could contain the root (CEO) or a mapping of id->Employee for quick lookup.

- Design Considerations:

- Storing direct reports: Either each Employee has a list of `directReports` (Employee objects or IDs). This makes traversals easy (top-down). If you also store `manager` field in each Employee, that's useful for upward traversal, but not strictly necessary if you have the top-down links.

- Ensure no cycles (in a well-formed org chart there shouldn't be any).

- If grouping is required (as in the interview example: grouping reports by department and experience), consider that as just organizing the output: e.g., `getAllReports(employee)` returns a list, then you can iterate and group by attributes (maybe using a data structure like `Map<Department, List<Employee>>`). You might incorporate that into the design by writing a method `getReportsGroupedBy(criteria)` or similar.

- **Example Query from prompt:** In one instance, they wanted to retrieve an employee's direct and indirect reports, grouped by experience and department. So an example solution: perform a DFS/BFS from that employee to collect all descendants. Then use two-level grouping: first group by experience level (say experience in years), then within each, group by department. This is a bit complex to implement on the fly, but you can describe it: traverse hierarchy to get list of reports, then use Java streams or just loops to build a nested map structure. The candidate in the example solved it with DFS and dynamic grouping function.
- **Depth:** Interviewers expect a well-structured approach rather than hacking it procedurally. Show that the hierarchy is naturally a tree and you can traverse it. Use recursion or a queue for traversal. If writing code, careful with null checks for leaf nodes. Also, mention edge cases: asking for reports of someone with none returns empty; the CEO's manager is null, etc. If they push further, discuss how you might extend for multiple reporting lines (matrix orgs) or managing huge orgs (maybe store in database), but typically not needed for new grad LLD. The key outcome is demonstrating an understanding of tree data structures through the lens of OOP (each node has children) and possibly using appropriate patterns (Composite pattern can be mentioned: treating Employee and Manager uniformly with a structure for subordinates is basically composite).

25. Design a Text Editor (Word Processor)

Why it's asked: A text editor design examines how you manage a large collection of objects (characters) and their formatting efficiently. It's often used to discuss design patterns like **Flyweight** (for sharing character styles) and to handle operations like insert/delete. Microsoft has asked this in the context of designing Word, for instance. While not extremely common at Amazon for entry-level, it is a known LLD exercise and rounds out understanding of performance and design principles.

What to cover:

- Core Features:

- **Document Model:** Represent the document as a sequence of characters, possibly grouped into lines or paragraphs. Each character can have formatting (font, size, bold/italic, etc.).
- **Editing Operations:** Support inserting and deleting characters at a cursor position. This involves shifting content and handling new lines if any.
- **Formatting:** Ability to apply styles to text (like making text bold, changing font). Each character or a block of text carries style information.
- **Displaying Text:** Not a focus of LLD usually, but you can mention that the editor would need to render the characters with the given styles.

- Entities:

- **Character** (or a generic **Glyph** if thinking like OOP in textbooks): stores a char value and possibly a reference to a **Style** (containing font, size, style attributes).
- **Style** (or **CharacterStyle**): includes `fontName`, `fontSize`, bold/italic flags, etc. Many characters can share one Style object if they have identical formatting.
- **Document** or **Editor** class: manages the sequence of Character objects (could be as simple as a list of Characters or lines). Provides methods like `insertChar(pos, char, style)` and `deleteRange(start, end)`, `getText()`.
- Maybe **Cursor** or **Caret** to track current insertion point, but that can be just an index within Document for simplicity.
- **Performance Concerns:** A naïve representation of each character with full style info is memory-intensive if the document is large and mostly uniform style. This is where **Flyweight pattern** comes in: share the style objects among characters. For example, all characters that are 12pt Arial share one Style instance

instead of each having their own copy. Mention that this drastically reduces memory usage when document has limited distinct styles even if it has millions of characters.

- Operations Implementation:

- **Insert:** At a given position, create a new Character with the typed letter and current style (which might be taken from context or user selection) and insert into the list. This could be $O(n)$ if using an array list (since characters after position move one index). Some designs might use a linked list or a gap buffer for efficiency, but you can keep it simple or mention if you know.

- **Delete:** Remove character(s) from the list, adjust any data structures accordingly.

- **Formatting:** If user selects a range and applies a style (say bold), you would update the Style of those characters. You might create a new Style object representing bold version of their current font and assign it to those characters. If using flyweight, ensure you reuse existing style objects if the same combination exists, or create and store a new one if not.

- **Design Patterns:** Flyweight is the main one to mention (characters share style intrinsics). Additionally, **Command pattern** could be discussed if we talk about undo/redo of edits (each edit is a Command that can be un-done). But undo/redo might be beyond the core ask unless specifically brought up.

- **Depth:** Focus on the data model and how it handles large data. Possibly toss in that even with 100k+ characters, storing them is fine, and operations are linear in the worst case which is acceptable for moderate use (advanced editors use gaps or ropes for efficiency, but that might be too much detail). Interviewers likely want to see you identify the memory versus performance trade-off and use of flyweight to optimize memory. They may also ask about how to read a line or specific position – which is trivial if you have the list of characters (just index into it or iterate until newline). So demonstrating traversal and understanding of text layout (rows/columns) can be good if relevant. Overall, highlight a clean separation: characters know their style, styles are reused, and the editor manipulates the characters.

Each of these design questions can go very deep, but as a new-grad/entry-level candidate, you should focus on getting the **class design and relationships correct**, and mention relevant **design patterns or principles** (SOLID) when appropriate. In an interview, clarify requirements first (ask what features to focus on), then outline your approach, and proceed to discuss the key classes and methods. Always be ready with a real example or two to illustrate your design working, and cite trade-offs or alternatives if time permits. By practicing these top 25 LLD problems – particularly the ones frequently noted in Amazon interviews (e.g. Parking Lot, LRU Cache, Amazon Locker, etc.) – you'll be well-prepared to handle object-oriented design questions in your interviews.

Sources:

1. Prashant Priyadarshi, "*Solving Top 10 Low Level Design (LLD) Interview Questions*" – Medium (Jul 2024)
2. Prashant (MockGym), Reddit discussion on Amazon LLD interviews (Sep 2025)
3. LeetCode Discuss, "*Frequently Asked Low-Level Design Questions*" (Jun 2024) 8 21
4. LeetCode Discuss, "*OOD Question: Design a Pizza Shop in Amazon interview*" (Jan 2025)
5. Reddit post by *Environmental-Fix428*, Amazon new grad interview experience (Mar 2025)
6. GeeksforGeeks, Amazon SDE-2 interview experience (Jul 2025)
7. Medium, "*Design In-Memory File System*" – Santanu Das (Feb 2021)
8. Reddit post, "*A detailed interview experience at Amazon - New grad*" (2025)

1 2 3 4 15 30 31 Solving Top 10 Low Level Design (LLD) Interview Questions | by Prashant Priyadarshi | Medium

<https://medium.com/@prashant558908/solving-top-10-low-level-design-llc-interview-questions-in-2024-302b6177c869>

5 6 7 8 9 10 11 12 13 14 16 17 18 19 20 21 22 23 24 25 26 27 28 29 Frequently Asked Low-Level Design (LLD) Questions - Discuss - LeetCode

[https://leetcode.com/discuss/post/5328221/Frequently-Asked-Low-Level-Design-\(LLD\)-Questions/](https://leetcode.com/discuss/post/5328221/Frequently-Asked-Low-Level-Design-(LLD)-Questions/)